

CS Study

Spring - 공통 예외 처리

목차

1

SpringBoot 기본 예외 처리

2

@ExceptionHandler

3

@ControllerAdvice

4

공통 예외 처리 위한 클래스

목차

1

SpringBoot 기본 예외 처리

2

@ExceptionHandler

3

@ControllerAdvice

4

공통 예외 처리 위한 클래스

SpringBoot 기본 예외 처리

SpringBoot 기본 예외 처리

- 아무런 예외 처리 코드 작성하지 않아도 **자동으로 예외 처리**
- **적절한 에러 페이지 또는 JSON 응답 반환**
- 이런 **예외 처리 위한 컨트롤러를 미리 등록**해 놓음 → **BasicExceptionHandler**

SpringBoot 기본 예외 처리

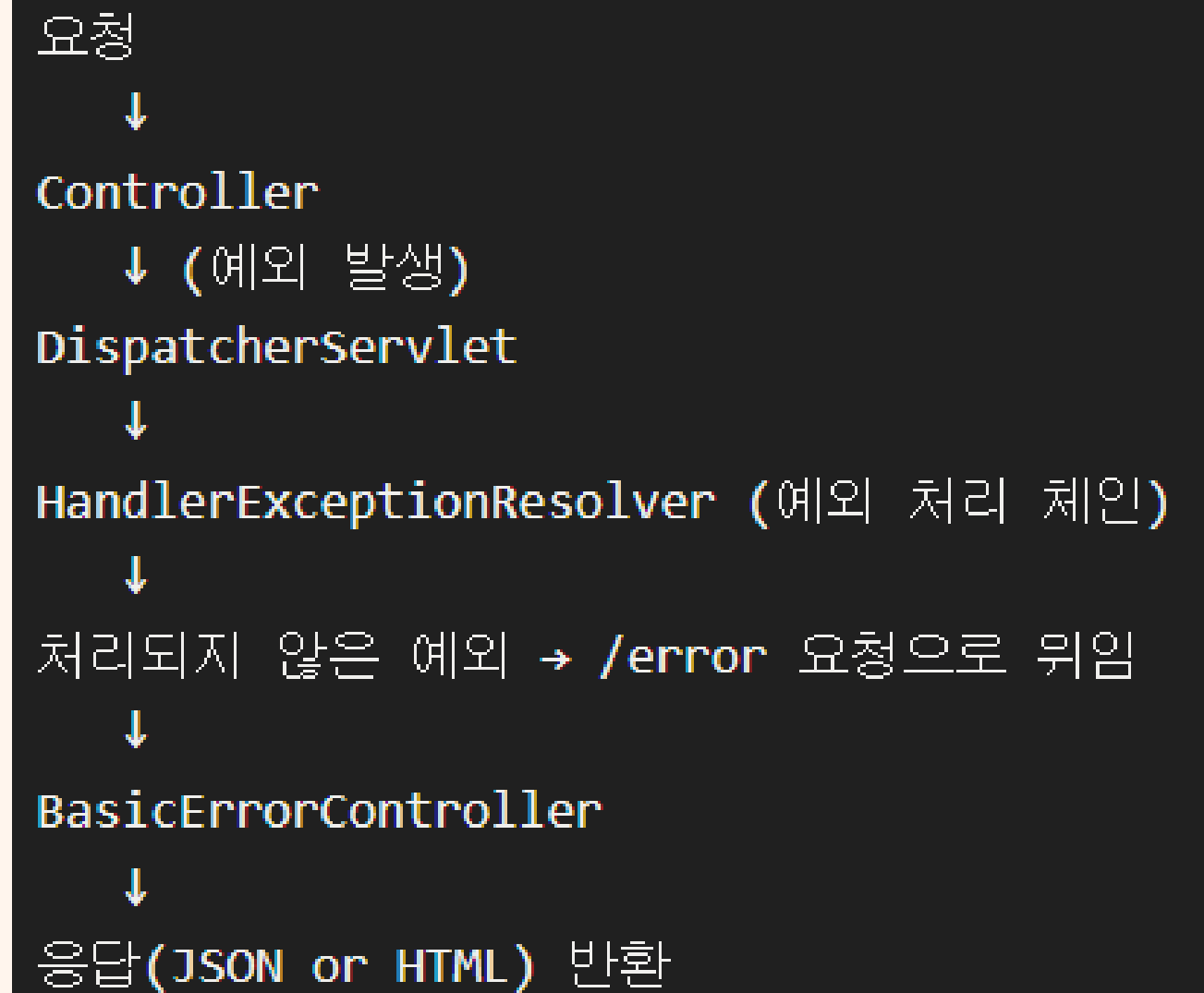
BasicExceptionHandler

- 아무도 예외 처리 안하면 **DispatcherServlet**이 **/error**로 이동
- **/error** 경로 처리하는 컨트롤러가 **BasicExceptionHandler**
- 요청 종류에 따라 다르게 결과를 반환 (HTML 요청, JSON 요청)

SpringBoot 기본 예외 처리

BasicErrorController 요청 흐름

- 별도의 예외 처리 하지 않으면 **WAS**까지
에러 전달
- /error로 **에러 요청을 다시 전달**
- **에러 컨트롤러 한번 더 호출**하는 것



SpringBoot 기본 예외 처리

BasicErrorController 요청 구분

- HTTP 요청의 **Accept** 헤더 값 보고 판단
- **브라우저 요청인지, API 요청인지** 구분
- 브라우저 요청인 경우
 - **Whitelabel Error Page** 반환
- API 요청인 경우
 - **기본 JSON** 응답 반환

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Thu Nov 06 15:47:30 KST 2025

There was an unexpected error (type=Not Found, status=404).

No static resource test.

org.springframework.web.servlet.resource.NoResourceFoundException: No static resource test.

at org.springframework.web.servlet.resource.ResourceHttpRequestHandler.handleRequest(ResourceHttpRequestHandler.java:585)

at org.springframework.web.servlet.mvc.HttpRequestHandlerAdapter.handle(HttpRequestHandlerAdapter.java:52)

at org.springframework.web.servlet.DispatcherServlet.doDispatch(DispatcherServlet.java:1089)

at org.springframework.web.servlet.DispatcherServlet.doService(DispatcherServlet.java:979)

at org.springframework.web.servlet.FrameworkServlet.processRequest(FrameworkServlet.java:1014)

at org.springframework.web.servlet.FrameworkServlet.doGet(FrameworkServlet.java:903)

at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:564)

at org.springframework.web.servlet.FrameworkServlet.service(FrameworkServlet.java:885)

at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:658)

at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:193)

at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:138)

```
{
  "timestamp": "2025-11-06T09:40:23.671+00:00",
  "status": 500,
  "error": "Internal Server Error",
  "path": "/api/v1/test"
}
```

SpringBoot 기본 예외 처리

BasicErrorController 한계점

- 일관성 없는 응답 format
- 부족한 정보
 - 예외 발생시 문제 파악 어려움
- 비즈니스 예외 구분 불가
 - WAS가 에러 받아 커스텀 예외도 500 처리
- HTML 페이지 노출
 - url만 알면 error 페이지 접근 가능

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Thu Nov 06 15:47:30 KST 2025

There was an unexpected error (type=Not Found, status=404).

No static resource test.

org.springframework.web.servlet.resource.NoResourceFoundException: No static resource test.

at org.springframework.web.servlet.resource.ResourceHttpRequestHandler.handleRequest(ResourceHttpRequestHandler.java:585)

at org.springframework.web.servlet.mvc.HttpRequestHandlerAdapter.handle(HttpRequestHandlerAdapter.java:52)

at org.springframework.web.servlet.DispatcherServlet.doDispatch(DispatcherServlet.java:1089)

at org.springframework.web.servlet.DispatcherServlet.doService(DispatcherServlet.java:979)

at org.springframework.web.servlet.FrameworkServlet.processRequest(FrameworkServlet.java:1014)

at org.springframework.web.servlet.FrameworkServlet.doGet(FrameworkServlet.java:903)

at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:564)

at org.springframework.web.servlet.FrameworkServlet.service(FrameworkServlet.java:885)

at jakarta.servlet.http.HttpServlet.service(HttpServlet.java:658)

at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:193)

at org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:138)



목차

1

SpringBoot 기본 예외 처리

2

@ExceptionHandler

3

@ControllerAdvice

4

공통 예외 처리 위한 클래스

@ExceptionHandler

@ExceptionHandler

- **컨트롤러 클래스 내부**에 작성
- **컨트롤러에서 발생한** 특정 예외 **지역적으로 처리**하는 annotation
- 코드를 작성한 **컨트롤러에서만 발생하는 예외만 처리**

@ExceptionHandler

```
@RestController
@RequestMapping("/users")
public class UserController {

    private final UserService userService;

    // 비즈니스 로직을 처리하는 기본 API
    @GetMapping("/{id}")
    public ResponseEntity<UserDto> getUserById(@PathVariable Long id) {
        // userService.findById(id)가 UserNotFoundException을 throw 할 수 있음
        UserDto user = userService.findById(id);
        return ResponseEntity.ok(user);
    }

    // -----
    // @ExceptionHandler를 통한 예외 처리
    @ExceptionHandler({UserNotFoundException.class})
    public ResponseEntity<ErrorResponse> handleUserNotFound(UserNotFoundException ex) {
        // 정의한 공통 오류 응답 DTO
        ErrorResponse response = new ErrorResponse("USER_NOT_FOUND", ex.getMessage());

        // 404 상태 코드와 ErrorResponse 본문을 담아 응답
        return new ResponseEntity<>(response, HttpStatus.NOT_FOUND);
    }
    // -----
}
```

@ExceptionHandler

@ExceptionHandler

- 어노테이션 속성으로 **처리할 예외 지정 가능**
- 해당 **예외 처리하는 메서드**에 붙임
- 예외 지정하지 않으면 **파라미터로 넘기는 예외 처리함**
- 지정한 예외와 파라미터로 넘기는 예외 **동일해야 함**

@ExceptionHandler

```
// @ExceptionHandler를 통한 예외 처리
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<ErrorResponse> handleUserNotFound(UserNotFoundException ex) {
    // 정의한 공통 오류 응답 DTO
    ErrorResponse response = new ErrorResponse("USER_NOT_FOUND", ex.getMessage());

    // 404 상태 코드와 ErrorResponse 본문을 담아 응답
    return new ResponseEntity<>(response, HttpStatus.NOT_FOUND);
}
```


@ExceptionHandler

@ExceptionHandler

- **에러 응답 자유롭게 다룰 수 있다**는 점에서 유연
- 정의한 **공통 응답에 맞춰 응답 가능**
- 반환하려는 body, 메시지 **예외별로 세밀하게 제공 가능**

@ExceptionHandler

```
// @ExceptionHandler를 통한 예외 처리
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<ErrorResponse> handleUserNotFound(UserNotFoundException ex) {
    // 정의한 공통 오류 응답 DTO
    ErrorResponse response = new ErrorResponse("USER_NOT_FOUND", ex.getMessage());

    // 404 상태 코드와 ErrorResponse 본문을 담아 응답
    return new ResponseEntity<>(response, HttpStatus.NOT_FOUND);
}
```

@ExceptionHandler

@ExceptionHandler 한계점

- 코드 중복
 - 작성한 컨트롤러에서만 발생하는 예외만 처리됨
 - 다른 컨트롤러에서도 같은 예외 처리 시 메서드 코드 중복
- 관심사 분리 위배
 - 예외 처리라는 **부가적인 코드가 컨트롤러 내부에 섞임**
 - 컨트롤러 클래스 **비대해지고 코드 지저분해짐**

목차

1

SpringBoot 기본 예외 처리

2

@ExceptionHandler

3

@ControllerAdvice

4

공통 예외 처리 위한 클래스

@ControllerAdvice

@ControllerAdvice

- @ExceptionHandler의 **코드 중복 문제 해결**할 수 있는 것
- 프로젝트 전역에서 발생하는 **예외 한 곳에서 모아서 처리**할 수 있게 하는 어노테이션
- **여러 컨트롤러에 대해 전역적으로 ExceptionHandler를 적용**해주는 것
- 모든 컨트롤러에 대해 횡단적으로 동작하는 **AOP 기반 예외 처리기**

@ControllerAdvice

@RestControllerAdvice

- @ControllerAdvice + @ResponseBody
- **HTTP 응답 본문** (Response Body)으로 반환
- **REST API 프로젝트**에서 주로 사용

```
@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@ControllerAdvice
@ResponseBody
public @interface RestControllerAdvice {
    ...
}

@Target(ElementType.TYPE)
@Retention(RetentionPolicy.RUNTIME)
@Documented
@Component
public @interface ControllerAdvice {
    ...
}
```

@ControllerAdvice

@ControllerAdvice 사용 방법

- 각 컨트롤러에 중복으로 존재했던 예외 처리 로직을 **별도의 예외 처리 클래스로 분리**
- 예외 처리 클래스에 **@RestControllerAdvice**를 붙여주면 됨
- 예외 처리 클래스에 **@ExceptionHandler** 붙은 예외 처리 메서드들 모아두면 됨
- 모든 컨트롤러에서 발생한 예외 모아둔 **예외 처리 메서드들에서 잡아 처리**

@ControllerAdvice

```
// 이 어노테이션 하나로, 모든 @RestController에서 발생하는 예외를 감시합니다.  
@RestControllerAdvice  
public class GlobalExceptionHandler {  
  
    // UserNotFoundException이 터지면 이 메서드가 잡습니다.  
    // (UserController, PostController 등 어느 컨트롤러에서 발생하든 상관없이!)  
    @ExceptionHandler(UserNotFoundException.class)  
    @ResponseStatus(HttpStatus.NOT_FOUND) // 404  
    public ErrorResponse handleUserNotFound(UserNotFoundException ex) {  
        return new ErrorResponse("USER_NOT_FOUND", ex.getMessage());  
    }  
}
```


@ControllerAdvice

@ControllerAdvice 적용 범위 설정

- AOP로 프로젝트 전체에 적용
- **특정 범위만 선택 가능**
- 예외 처리 **모듈별로 분리 가능**

```
@ControllerAdvice(  
    basePackages = "com.example.api.user",    // 특정 패키지만  
    assignableTypes = {UserController.class}, // 특정 클래스만  
    annotations = RestController.class       // 특정 어노테이션만  
)  
public class UserExceptionHandler { ... }
```

@ControllerAdvice

@ControllerAdvice 우선 순위

- 전역 예외 처리 클래스 여러 개라면?
- **@Order** 또는 **Ordered**로 순서 지정 가능
- 순서 지정하지 않으면?
- ControllerAdvice **임의의 순서로 호출**

```
@Order(1)
@RestControllerAdvice
public class GlobalSecurityExceptionHandler { ... }

@Order(2)
@RestControllerAdvice
public class GlobalBusinessExceptionHandler { ... }
```

목차

1

SpringBoot 기본 예외 처리

2

@ExceptionHandler

3

@ControllerAdvice

4

공통 예외 처리 위한 클래스

공통 예외 처리 위한 클래스

ErrorCode (인터페이스)

- **항상 동일한 오류 코드와 메시지** 반환하도록 강제
- 에러 코드와 메시지를 **한 곳에서 관리하고 유지보수**

```
public interface ErrorCode {  
  
    String name();  
    HttpStatus getHttpStatus();  
    String getMessage();  
}
```

공통 예외 처리 위한 클래스

ErrorCode (구현체)

- **필요한 상황에 맞는** 에러 코드 클래스 구현
- **enum 클래스**로 구현

```
@Getter
@RequiredArgsConstructor
public enum CommonErrorCode implements Errorcode {

    INVALID_PARAMETER(HttpStatus.BAD_REQUEST, "Invalid parameter included"),
    RESOURCE_NOT_FOUND(HttpStatus.NOT_FOUND, "Resource not exists"),
    INTERNAL_SERVER_ERROR(HttpStatus.INTERNAL_SERVER_ERROR, "Internal server error"),
    ;

    private final HttpStatus httpStatus;
    private final String message;
}
```

공통 예외 처리 위한 클래스

UserException

- **상황에 맞는 ErrorCode 담아**
던지기 위한 예외 클래스
- **ErrorCode를 필드로 가짐**
- **RuntimeException 상속**

```
@Getter
@RequiredArgsConstructor
public class RestApiException extends RuntimeException {

    private final ErrorCode errorCode;

}
```

공통 예외 처리 위한 클래스

ErrorResponse

- 에러 코드, 메시지 담은 **공통 에러 응답 클래스**
- **일관되고 표준화된 형식**의 오류 JSON 반환
- **프론트엔드와 협업 시** 필요

```
@Getter
@AllArgsConstructor
public class ApiErrorResponse {
    private final String code;
    private final String message;
    private final LocalDateTime timestamp = LocalDateTime.now();
}
```